TEK-UP — Private Higher School of Technologies & Engineering

Programmation Orientée Objet Java

Résumé Complet du Cours + Corrections des Exercices

Chapitres couverts : POO Concepts • Héritage • Polymorphisme • Classes Abstraites • Interfaces • Exceptions

TABLE DES MATIÈRES

Chapitre 1 — Introduction à la POO

Chapitre 3 — Héritage et Polymorphisme

Chapitre 4 — Classes Abstraites et Interfaces

Chapitre 6 — Les Exceptions Java

Corrections — Exercices — Encapsulation, Héritage, Polymorphisme, Exceptions

CHAPITRE 1

Introduction à la POO

Concepts fondamentaux

1.1 — Les 4 Piliers de la POO

Encapsulation	Regrouper attributs + méthodes dans un objet. Contrôler l'accès via les modificateurs (private, protected, public).
Abstraction	Masquer la complexité, exposer uniquement l'interface utile.
Héritage	Une classe fille hérite de la classe mère (extends). Relation "est-un" (is-a).
Polymorphisme	Un objet peut prendre plusieurs formes. Surcharge (statique) et redéfinition (dynamique).

1.2 — Niveaux de Visibilité

Modificateur	Description	Sous-classes	Package	Extérieur
private	Classe elle-même uniquement	✗ Sous-classes	✗ Package	✗ Partout
(default)	Classe + même package	✗ Sous-classes ext.	✓ Package	✗ Hors package
protected	Classe + sous-classes + package	✓ Sous-classes	✓ Package	✗ Hors package
public	Partout dans l'application	✓	✓	✓

CHAPITRE 3

Héritage et Polymorphisme

extends • super • @Override • instanceof

3.1 — Syntaxe de l'Héritage

Le mot-clé **extends** établit la relation d'héritage. La classe fille récupère tous les membres **public** et **protected** de la classe mère.

```
// Classe mère
public class Animal {
protected String nom; // accessible par sous-classes
public Animal(String nom) {
this.nom = nom;
}
public void manger() {
System.out.println(nom + " mange.");
}
}

// Classe fille
public class Chien extends Animal {
private String race;

public Chien(String nom, String race) {
super(nom); // OBLIGATOIRE — 1ère instruction
this.race = race;
}

public void aboyer() {
System.out.println(nom + " aboie !");
}
}
```

3.2 — Redéfinition (Override)

Règle	Explication
Signature identique	Même nom + mêmes paramètres que la méthode parente.
Visibilité non restrictive	Ne peut pas passer de public à protected.
Covariance	Le type de retour peut être une sous-classe du type original.
Interdit sur final/static	Impossible de redéfinir une méthode final ou static.

```
public class Oiseau extends Animal {
@Override
public void manger() {
super.manger(); // optionnel — appelle la version parente
System.out.println("Picore des graines.");
}
}
```

3.3 — Le mot-clé final

```
public final class Securite { } // 1. Interdit l'héritage de cette classe

public class Compte {
public final void valider() { } // 2. Interdit la redéfinition de cette méthode
}
```

3.4 — Polymorphisme Dynamique (Runtime)

La JVM détermine à l'exécution quelle version de la méthode appeler, selon le **type réel** de l'objet (pas le type déclaré).

```
Animal a1 = new Chien("Rex", 3);
Animal a2 = new Chat("Felix", 2);

a1.manger(); // -> Chien.manger() (résolution à l'exécution)
a2.manger(); // -> Chat.manger()

// ERREUR de compilation : Animal ne connait pas aboyer()
// a1.aboyer();

// Pour appeler aboyer(), il faut un cast :
if (a1 instanceof Chien chien) { // Pattern Matching (Java 16+)
chien.aboyer();
}
```

3.5 — Polymorphisme Statique (Surcharge / Overloading)

Plusieurs méthodes de **même nom** mais de **signatures différentes**. Le compilateur choisit la bonne méthode selon les types des arguments.

```
public class Calculatrice {
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
int add(int a, int b, int c) { return a + b + c; }
}
```

3.6 — Collections Polymorphes

```
List<Animal> zoo = new ArrayList<>();
zoo.add(new Chien("Rex"));
zoo.add(new Chat("Felix"));

for (Animal a : zoo) {
a.manger(); // appel résolu dynamiquement (Runtime)
}
```

CHAPITRE 4

Classes Abstraites et Interfaces

abstract • interface • implements

4.1 — Classe Abstraite

Une classe abstraite est un **modèle incomplet** : non instanciable directement, elle peut mélanger méthodes abstraites (sans corps) et méthodes concrètes (avec corps).

```
public abstract class Forme {
protected String couleur;

public Forme(String couleur) {
this.couleur = couleur;
}

// Méthode abstraite — les sous-classes DOIVENT l'implémenter
public abstract double calculerAire();
public abstract double calculerPerimetre();

// Méthode concrète — héritée telle quelle
public void afficherInfo() {
System.out.println("Forme : " + couleur + ", Aire=" + calculerAire());
}
}

public class Cercle extends Forme {
private double rayon;

public Cercle(String c, double r) { super(c); this.rayon = r; }

@Override
public double calculerAire() { return Math.PI * rayon * rayon; }
@Override
public double calculerPerimetre() { return 2 * Math.PI * rayon; }
}
```

4.2 — Interface

Une interface définit un **contrat** (capacité "peut faire"). Toutes ses méthodes sont implicitement **public abstract**, ses constantes sont **public static final**. Une classe peut implémenter **plusieurs interfaces**.

```
public interface Volant {
double VITESSE_MAX = 1000.0; // public static final implicite

void decoller(); // public abstract implicite
void atterrir();
void voler(double altitude);

// Depuis Java 8 : méthodes default et static autorisées
default String getInfo() { return "Objet volant"; }
}

public class Avion implements Volant {
@Override public void decoller() { System.out.println("Décollage."); }
@Override public void atterrir() { System.out.println("Atterrissage."); }
@Override public void voler(double alt) { System.out.println("Vol à " + alt + "m"); }
}

// Implémentation multiple
public class Canard implements Volant, Nageant, Marchant { ... }
```

4.3 — Classe Abstraite vs Interface (Comparatif)

Critère	Classe Abstraite	Interface
Instanciable	Non	Non
Constructeur	Oui	Non
Attributs d'état	Variables d'instance (private, protected…)	Constantes uniquement (public static final)
Méthodes	Abstraites + Concrètes	Abstraites par défaut + default + static (Java 8+)
Héritage	extends (unique)	implements (multiple)
Usage sémantique	Relation "est un" — partage de code	Capacité "peut faire" — contrat / API

4.4 — Interface Fonctionnelle et Lambda (Java 8+)

```
@FunctionalInterface
public interface Calculable {
double calculer(double a, double b); // exactement 1 méthode abstraite
}

// Implémentation via Lambda
Calculable add = (a, b) -> a + b;
Calculable mul = (a, b) -> a * b;

System.out.println(add.calculer(5, 3)); // 8.0
```

CHAPITRE 6

Les Exceptions Java

try • catch • finally • throw • throws

6.1 — Hiérarchie des Exceptions

Type	Hiérarchie	Traitement requis
Error	Hérite de java.lang.Error	Ne pas attraper — problèmes JVM graves (StackOverflowError, OutOfMemoryError)
Checked Exception	Hérite de Exception (pas RuntimeException)	OBLIGATOIRE : handle (try-catch) ou declare (throws). Ex: IOException, SQLException
Unchecked Exception	Hérite de RuntimeException	Optionnel. Erreurs de logique. Ex: NullPointerException, ArithmeticException

6.2 — Structure try-catch-finally

```
try {
// Code susceptible de lever une exception
int result = 10 / 0; // ArithmeticException
String s = null;
s.length(); // NullPointerException

} catch (ArithmeticException e) { // Capture la + spécifique d'abord !
System.out.println("Division par zero : " + e.getMessage());

} catch (NullPointerException e) {
System.out.println("Reference nulle : " + e.getMessage());

} catch (Exception e) { // Capture tout le reste (+ général en dernier)
System.out.println("Erreur : " + e.getMessage());

} finally {
// TOUJOURS exécuté (sauf System.exit())
System.out.println("Nettoyage des ressources.");
}
```

Règle d'or sur l'ordre des catch : Du plus spécifique (sous-classe) au plus général (super-classe). Sinon, erreur de compilation "Unreachable Code".

6.3 — throw et throws

```
// throw : LANCER une instance d'exception (action)
// throws : DÉCLARER qu'une méthode peut lever une exception

public void registerUser(String username, int age) throws Exception {
if (username == null) {
throw new IllegalArgumentException("Username ne peut pas etre null");
// IllegalArgumentException est Unchecked -> pas besoin de throws
}
if (age < 18) {
throw new Exception("L utilisateur doit avoir 18+");
// Exception est Checked -> OBLIGATOIRE de déclarer throws
}
}
```

6.4 — Exceptions Personnalisées (Custom)

```
// Checked custom exception
public class SoldeInsuffisantException extends Exception {
public SoldeInsuffisantException(String message) {
super(message);
}
}

// Unchecked custom exception
public class DeviseIncompatibleException extends RuntimeException {
public DeviseIncompatibleException(String message) {
super(message);
}
}

// Utilisation
public void retirer(double montant) throws SoldeInsuffisantException {
if (montant > solde) {
throw new SoldeInsuffisantException("Solde insuffisant : " + solde);
}
solde -= montant;
}
```

6.5 — Multi-catch et Exceptions Courantes

```
// Multi-catch (Java 7+) — types séparés par |
try {
Files.readAllLines(path);
database.connect();
} catch (IOException | SQLException e) {
System.out.println("Erreur systeme : " + e.getMessage());
}
```

Exception	Type	Cause fréquente

NullPointerException	Unchecked	Accès sur une référence null
ArithmeticException	Unchecked	Division entière par zéro
ArrayIndexOutOfBoundsException	Unchecked	Index tableau invalide
NumberFormatException	Unchecked	Conversion String→nombre invalide
ClassCastException	Unchecked	Cast incompatible (Cat casté en Dog)
IOException	Checked	Erreur lecture/écriture fichier/réseau
FileNotFoundException	Checked	Fichier introuvable (sous-classe IOException)
SQLException	Checked	Erreur base de données

CORRECTIONS DES EXERCICES

Exercice Exceptions 1 — Division avec gestion de zéro

```
import java.util.Scanner;

public class Ex1Division {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Entrez le premier nombre : ");
int a = sc.nextInt();
int b = 0;
do {
System.out.print("Entrez le deuxième nombre (≠ 0) : ");
b = sc.nextInt();
if (b == 0) {
System.out.println("Erreur : la division par zéro n'est pas autorisée.");
}
} while (b == 0);
System.out.println("Résultat : " + a + " / " + b + " = " + (double) a / b);
}
}
```

Exercice Exceptions 2 — Conversion String → entier

```
import java.util.Scanner;

public class Ex2Conversion {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int nombre = 0;
boolean valide = false;
while (!valide) {
System.out.print("Entrez un entier : ");
String saisie = sc.nextLine();
try {
nombre = Integer.parseInt(saisie);
valide = true;
} catch (NumberFormatException e) {
System.out.println("Erreur : \"" + saisie + "\" n'est pas un entier valide.");
}
}
System.out.println("Entier saisi : " + nombre);
}
}
```

Exercice Exceptions 3 — Année de naissance / Calcul de l'âge

```
import java.util.Scanner;

public class Ex3Age {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int anneeActuelle = 2025;
int annee = 0;
boolean valide = false;
while (!valide) {
System.out.print("Entrez votre année de naissance : ");
try {
annee = Integer.parseInt(sc.nextLine());
if (annee > anneeActuelle || annee < 1900) {
System.out.println("Erreur : année non valide (" + annee + ").");
} else {
valide = true;
}
} catch (NumberFormatException e) {
System.out.println("Erreur : veuillez entrer un nombre entier.");
}
}
System.out.println("Votre âge : " + (anneeActuelle - annee) + " ans");
}
}
```

Exercice Exceptions 4 — Compte bancaire avec exception personnalisée

```
// Exception personnalisée
public class SoldeInsuffisantException extends Exception {
public SoldeInsuffisantException(String message) {
super(message);
}
}

// Classe CompteBancaire
public class CompteBancaire {
private double solde;

public CompteBancaire(double solde) { this.solde = solde; }

public void retirer(double montant) throws SoldeInsuffisantException {
if (montant > solde) {
throw new SoldeInsuffisantException(
"Solde insuffisant : disponible=" + solde + ", demande=" + montant);
}
solde -= montant;
System.out.println("Retrait de " + montant + " effectue. Solde : " + solde);
}

public static void main(String[] args) {
CompteBancaire compte = new CompteBancaire(500);
try {
compte.retirer(200);
compte.retirer(400); // Dépasse le solde restant
} catch (SoldeInsuffisantException e) {
System.out.println("Erreur : " + e.getMessage());
}
}
}
```

Exercice Monnaie — Classe Monnaie + DeviseIncompatible

```
// Exception personnalisée
public class DeviseIncompatible extends Exception {
public DeviseIncompatible(String message) {
super(message);
}
}

// Classe Monnaie
public class Monnaie {
private double valeur;
private String devise;

public Monnaie(double valeur, String devise) {
this.valeur = valeur;
this.devise = devise;
}

public double getValeur() { return valeur; }
public String getDevise() { return devise; }

public Monnaie ajouter(Monnaie m) throws DeviseIncompatible {
if (!this.devise.equals(m.devise)) {
throw new DeviseIncompatible(
"Devises incompatibles : " + this.devise + " vs " + m.devise);
}
return new Monnaie(this.valeur + m.valeur, this.devise);
}

@Override
public String toString() {
return valeur + " " + devise;
}

public static void main(String[] args) {
Monnaie m1 = new Monnaie(100, "EUR");
Monnaie m2 = new Monnaie(50, "EUR");
Monnaie m3 = new Monnaie(30, "USD");
try {
System.out.println(m1.ajouter(m2)); // 150.0 EUR
System.out.println(m1.ajouter(m3)); // DeviseIncompatible
} catch (DeviseIncompatible e) {
System.out.println("Erreur : " + e.getMessage());
}
}
}
```

Exercice Java 1 — Encapsulation : Classe Livre

```
public class Livre {
private String titre;
private String auteur;
private boolean disponible;

public Livre(String titre, String auteur, boolean disponible) {
this.titre = titre;
this.auteur = auteur;
this.disponible = disponible;
}

public String getTitre() { return titre; }
public String getAuteur() { return auteur; }
public boolean isDisponible() { return disponible; }
public void setDisponible(boolean d) { this.disponible = d; }

public void emprunter() {
if (disponible) {
disponible = false;
System.out.println("Livre emprunte : " + titre);
} else {
System.out.println("Livre non disponible : " + titre);
}
}

public void retourner() {
disponible = true;
System.out.println("Livre retourne : " + titre);
}

@Override
public String toString() {
return titre + " (" + auteur + ") - " + (disponible ? "Disponible" : "Emprunte");
}
}
```

Exercice Java 2 — Encapsulation : CompteBancaire

```
public class CompteBancaire {
private String numeroCompte;
private double solde;

public CompteBancaire(String numeroCompte, double solde) {
this.numeroCompte = numeroCompte;
this.solde = solde;
}

public String getNumeroCompte() { return numeroCompte; }
public double getSolde() { return solde; }

public void deposer(double montant) {
if (montant > 0) { solde += montant; }
System.out.println("Depot : " + montant + " | Solde : " + solde);
}

public void retirer(double montant) {
if (montant > 0 && montant <= solde) {
solde -= montant;
System.out.println("Retrait : " + montant + " | Solde : " + solde);
} else {
System.out.println("Solde insuffisant ou montant invalide.");
}
}
}
```

Exercice Java 3 — Héritage : Vehicule / Voiture / Moto

```
public class Vehicule {
private String marque;
private double vitesseMax;

public Vehicule(String marque, double vitesseMax) {
this.marque = marque;
this.vitesseMax = vitesseMax;
}

public void afficherInfo() {
System.out.println("Marque : " + marque + " | Vitesse max : " + vitesseMax + " km/h");
}
}

public class Voiture extends Vehicule {
private int nombrePortes;

public Voiture(String marque, double vitesseMax, int nombrePortes) {
super(marque, vitesseMax);
this.nombrePortes = nombrePortes;
}

@Override
public void afficherInfo() {
super.afficherInfo();
System.out.println("Nombre de portes : " + nombrePortes);
}
}

public class Moto extends Vehicule {
private double cylindree;

public Moto(String marque, double vitesseMax, double cylindree) {
super(marque, vitesseMax);
this.cylindree = cylindree;
}

@Override
public void afficherInfo() {
super.afficherInfo();
System.out.println("Cylindrée : " + cylindree + " cc");
}
}
```

Exercice Java 4 — Héritage : Employe / Manager

```
public class Employe {
private String nom;
private double salaire;

public Employe(String nom, double salaire) {
this.nom = nom;
this.salaire = salaire;
}

public double calculerSalaire() { return salaire; }

public String toString() {
return nom + " -> Salaire : " + calculerSalaire();
}
}

public class Manager extends Employe {
private double prime;

public Manager(String nom, double salaire, double prime) {
super(nom, salaire);
this.prime = prime;
}

@Override
public double calculerSalaire() {
return super.calculerSalaire() + prime;
}
}
```

Exercice Java 5 — Polymorphisme : Système de Paiement

```
public class Paiement {
public void effectuerPaiement(double montant) {
System.out.println("Paiement effectue de " + montant + "EUR");
}
}

public class CarteBancaire extends Paiement {
@Override
public void effectuerPaiement(double montant) {
System.out.println("Paiement de " + montant + "EUR effectue par carte bancaire.");
}
}

public class PayPal extends Paiement {
@Override
public void effectuerPaiement(double montant) {
System.out.println("Paiement de " + montant + "EUR effectue via PayPal.");
}
}

// Main — usage polymorphe
Paiement p1 = new CarteBancaire();
Paiement p2 = new PayPal();
p1.effectuerPaiement(49.99);
p2.effectuerPaiement(120.00);
```

Exercice Java 6 — Polymorphisme : Animal / Chien / Chat

```
public class Animal {
public void faireSon() {
System.out.println("Cet animal fait un bruit.");
}
}

public class Chien extends Animal {
@Override
public void faireSon() {
System.out.println("Le chien aboie : Woof!");
}
}

public class Chat extends Animal {
@Override
public void faireSon() {
System.out.println("Le chat miaule : Miaou!");
}
}
```

Projet : Gestion d'une Banque (Account / SavingsAccount / CurrentAccount)

```
// Classe abstraite Account
public abstract class Account {
private String accountNumber;
private double balance;
private String ownerName;

public Account(String accountNumber, double balance, String ownerName) {
this.accountNumber = accountNumber;
this.balance = balance;
this.ownerName = ownerName;
}

private void setBalance(double balance) { this.balance = balance; }
public double getBalance() { return balance; }
public String getOwnerName() { return ownerName; }
public String getAccountNumber() { return accountNumber; }

public void deposit(double amount) {
if (amount > 0) { balance += amount; }
System.out.println("Depot : " + amount + " | Solde : " + balance);
}

public void withdraw(double amount) {
if (amount > 0 && amount <= balance) {
balance -= amount;
System.out.println("Retrait : " + amount + " | Solde : " + balance);
} else {
System.out.println("Retrait impossible : solde insuffisant.");
}
}
}
```

```
// SavingsAccount
public class SavingsAccount extends Account {
private double interestRate;

public SavingsAccount(String num, double balance, String owner, double rate) {
super(num, balance, owner);
this.interestRate = rate;
}

public void applyInterest() {
double interest = getBalance() * interestRate / 100;
deposit(interest);
System.out.println("Interet applique : " + interest);
}
}
```

```
// CurrentAccount
public class CurrentAccount extends Account {
private double overdraftLimit;

public CurrentAccount(String num, double balance, String owner) {
super(num, balance, owner);
}

public void setOverdraftLimit(double limit) { this.overdraftLimit = limit; }

@Override
public void withdraw(double amount) {
if (amount > 0 && amount <= getBalance() + overdraftLimit) {
// Autorise le découvert dans la limite
System.out.println("Retrait (avec decouvert possible) : " + amount);
} else {
System.out.println("Retrait depasse la limite de decouvert.");
}
}
}
```

```
// Classe Bank – polymorphisme
import java.util.ArrayList;
import java.util.List;

public class Bank {
private List<Account> accounts = new ArrayList<>();

public void addAccount(Account a) { accounts.add(a); }

public void processDeposit(Account a, double amount) { a.deposit(amount); }
public void processWithdraw(Account a, double amount) { a.withdraw(amount); }
public void showBalance(Account a) {
System.out.println(a.getOwnerName() + " -> Solde : " + a.getBalance());
}
}
```

Projet : Système de Paiement (Interface PaymentMethod)

```
// Interface
public interface PaymentMethod {
void processPayment(double amount);
String getPaymentName();
}

// Classe de base
public class Payment {
private String ownerName;
public Payment(String ownerName) { this.ownerName = ownerName; }
public String getOwnerName() { return ownerName; }
}
```

```
// CreditCardPayment
public class CreditCardPayment extends Payment implements PaymentMethod {
private String cardNumber;

public CreditCardPayment(String owner, String cardNumber) {
super(owner);
this.cardNumber = cardNumber;
}

@Override
public void processPayment(double amount) {
System.out.println("[Carte] " + getOwnerName() +
" paie " + amount + "EUR (carte : " + cardNumber + ")");
}

@Override
public String getPaymentName() { return "Carte Bancaire"; }
}
```

```
// PayPalPayment
public class PayPalPayment extends Payment implements PaymentMethod {
private String email;

public PayPalPayment(String owner, String email) {
super(owner);
this.email = email;
}

@Override
public void processPayment(double amount) {
System.out.println("[PayPal] " + getOwnerName() +
" paie " + amount + "EUR (email : " + email + ")");
}

@Override
public String getPaymentName() { return "PayPal"; }
}
```

```
// Application — polymorphisme via interface
public class Application {
public static void main(String[] args) {
List<PaymentMethod> payments = new ArrayList<>();
payments.add(new CreditCardPayment("Alice", "1234-5678-9012"));
payments.add(new PayPalPayment("Bob", "bob@email.com"));

for (PaymentMethod p : payments) {
System.out.println("Mode : " + p.getPaymentName());
p.processPayment(99.99);
}
}
}
```